

Bevezetés a Rust programozásba

Gembela Gergely

2. óra: Mutability, control flow and syntax
(Mit ismerünk máshonnan?)

<https://github.com/bmeaut/VIAUBXAV090>

Safe/unsafe előzetes

- A Rust egy *safe* nyelv, alapértelmezett módban ha egy kódrészlet fordítható, nem tartalmaz **memóriabiztonsággal** kapcsolatos hibát
 - Ez remek, de számos megkötést eredményez (pl. nem lehetséges *nyers* pointert dereferálni)
- A garanciák feladásával valamivel nagyobb szabadságot kapunk
 - Ezt a módot hívják *unsafe*-nek
 - Ekkor sem tehetünk meg bármit

Control flow

- A tárgy feltételezi, hogy tudtok C++-ban programozni, így itt csak röviden tekintjük át a Rust control flow alapvető elemeit
- A legtöbb elemnek van C++ megfelelője
 - Nem érdemes 1:1 leképezni ezeket C++ portolása esetén
 - Fontos különbség a string/char kezelése
 - Rust-ban a karakterek és a stringek is Unicode karakterek tárolására vannak felkészítve, nem 8 bites egy karakter (UTF-8)

Control flow

- Egyszerű pointereket (safe kódban) ritkán használunk, a referenciák viszont gyakoriak
- A céljuk ugyanaz, egy erőforrás címét tárolják, viszont
 - A referenciákról a Rust fordító fordítás során bizonyítja, hogy helyesek
 - A pointerokről nem, viszont hasznosak pl. alacsonyabb szintű műveletekre, és más nyelvekbe való híváskor

Függvények

- Jellemzően: `(pub)? fn name<T, ...>(args...) -> RetType {...}`
 - Nincsen C++-hoz hasonló overloading
- Ez egy remek alkalom a Rust Reference bemutatására
 - [Functions \(The Rust Reference\)](#).

if Expression

- A legtöbb nyelvhez hasonló szintaxis, viszont
 - nincs `()`
 - kötelező a `{ }`

```
if condition {  
    //condition igaz  
}  
else {  
    //különben  
}
```

Loopok

- **while** loop

```
while n < 42{  
    //...  
    n+=1;  
}
```

- **for** loop (magasabb szintű nyelvek **foreach** loopjához hasonló)
 - Iterátorok „végigjárásához” (a C++ **for(...;...;...)** loopja más)

```
for it in iterable{  
    //...  
}
```

És az igazi loop

```
let mut n = 3;  
loop {  
  
    if n < 21{  
        n = 21;  
        continue;  
    }  
  
    if n == 42{  
        break;  
    }  
  
    n += 1;  
}
```

Akár több is lehet

```
'outer: loop {  
    println!("outer...");  
  
    'inner: loop {  
        'forever: while 41 < 42{  
            //...  
        }  
        break 'outer;  
    }  
}
```

Pattern matching

- Több helyen is megjelenik a nyelvben
 - `match` alternatívákban
 - a `let` statementben
 - a `for` loopok fejlécében
- Vannak minták amik biztosan illeszkednek (irrefutable, pl. `let`, `for`)
- Vannak amik nem szükségesen illeszkednek (refutable, pl. `if let`, `while let`, `let...else`)

Pattern matching (`let`)

- Magányos `let ... = ...;` esetben csak biztosan illeszkedő mintát használhatunk, különben fordítási hibát kapunk

```
let (x, y, z) = (3.0, 42, "asdf");  
let (x1, ..) = (3.0, 42, "asdf");  
let (x2, _ , z2) = (3.0, 42, "asdf");
```

- Ugyanezt megtehetjük tetszőleges szinten
- Enumok, struktúrák, tuple-k is lehetnek egymásba ágyazva

Pattern matching (**match**)

- *Hasonlít* egyéb nyelvek **switch** konstrukciójára

```
match pontszám {  
    0..50 => println!("elégtelen"),  
    50..60 => println!("elégséges"),  
    60..75 => println!("közepes"),  
    75..85 => println!("jó"),  
    85..=100 => println!("jeles"),  
    _ => println!("nem_teljesítette")  
}
```

Pattern matching (`if let...`)

- Használjuk ezt amikor a match kényelmetlen lenne, mert csak egy esetre vagyunk kíváncsiak (pl. opcionális értékek kicsomagolásakor)
- Az enum típus bevezetésekor visszatérünk rá, hogy ez miért jó

```
if let Some(actual_value) = optional_val{  
    actual_value.//...  
}
```

- Hasonlóan visszatérünk majd a `let-else` és `while let` nyelvi elemekre is, a lényegük hasonló
- Nézzünk meg példákat a mellékelt crate-ben

Ternary expression?

Rust-ban minden kifejezés (kivéve a for/while loopokat) kiértékelhető értékke, ezért ami C++-ban:

```
const auto res = condition ? val_if_true : val_if_false;
```

Rust-ban:

```
let res = if condition {val_if_true} else {val_if_false};
```

Értéket eredményező kifejezések

- Ehhez hasonlóan használható a `match` is:

```
let res = match n {  
  42 => 3,  
  3  => 42,  
  _  => n  
};  
println!("{}", res);
```

Értéket eredményező kifejezések

- Sőt, akár loopok is

```
let result = loop {  
  //...  
  break 42;  
}
```

Range-ek

- Saját szintaxissal hozhatók létre tartományok:
- `1..100` | `1..=100` (exkluzív és inkluzív, az első elemre mindig zárt)
- Használhatók iterációhoz

```
for i in 1..100 { // ekvivalens ezzel: std::ops::Range {start: 1, end: 100}  
    //...  
}
```

- `..42` | `..` | `..=42` is valid tartomány, de nem használható iterációhoz, mert nincs kezdő/végpont `42..` pedig //TODO

Kommentek

- A C/C++ nyelvekhez hasonló kommentek használatosak
- A nyelv egyik beépített szolgáltatása a `rustdoc`
 - A dokumentációs kommentek `///`-el kezdődnek
 - Markdown formázást támogatnak
 - A dokumentáció attribútumokkal kontrollálható

Változók

- Minden változó alapértelmezetten immutable (`let <varname> ...`)
 - Ez sok esetben egyszerűsíti:
 - az optimalizációt
 - a biztonság biztosítását
 - a konkurenciakezelést
 - De van lehetőség mutable változót létrehozni (`let mut ...`)

Legyen mégis mutable

Alaphelyzetben:

```
let x = 42;  
x = 41; //fordítási hiba (cannot assign twice)
```

Legyen mutable:

```
let mut x = 41;  
x = 42; //:)
```

Copy vs Move

- C++-ban sok esetben értékadások/paraméterek copy ctor hívást eredményeznek
 - C++-ban van `std::move(...)`, de nagy különbségek vannak
 - Rust-ban másolás csak a `Copy` traitet implementáló típusoknál lehetséges (és bitenkénti másolást jelent)
 - Ha nincs implementálva, move szemantikát használ a fordító
 - Ha egy típusra implementálva van a `Clone` (kb. copy ctor, saját logikát futtat), kézzel használhatjuk

Konstansok

- Lényegében elnevezett értékek
 - A `const` kulcsszóval deklarálhatók
 - Típusuk megadása minden esetben kötelező
 - Bármely, akár globális scope-ban is felvehetők
 - Csak *konstans kifejezés* adható értékként, pl.

```
const A_VALASZ = 2 * 21;
```

Shadowing

- A C++-ban megszokottakkal szemben a változók közti shadowing engedélyezett, ezért
 - Itt n első ránézésre „mutable”

```
let n = 42;  
println!("n1: {n}");  
let n = 43;  
println!("n2: {n}");
```

- Konstansokkal és statikus értékekkel nem tehetjük meg

Makrók

- Komplex makrókat definiálhatunk, rengeteg funkció használja őket
- A makrók okosabbak mint C/C++-ban
 - Elsősorban kódDuplikáció kiküszöbölésére használjuk őket
- A feloldás eredményét pl RustRover-ben láthatjuk

Makrók definiálása

- Deklaratív eset (mintaillesztés jellegű):

```
macro_rules! m1 {  
    ($name:designator) => {  
        //ez lesz belőle  
    }  
}
```

- Procedurális eset (TokenStream -> TokenStream):
 - Saját crate szükséges a definíciójuknak
 - Visszatérünk rájuk a crate-ek tárgyalása után

Példák makrók használatára

- `println!` (ami feloldja a formázást, típushelyes kódot generál)
- `vec![...]`
- Serde és egyéb serializáció
- ORM-ek
- Webszerver könyvtárak

Attribútumok

- A C# attribútumokhoz/Java annotációkhoz hasonló konstrukció
 - [ECMA-335](#) + [ECMA-334](#) (C# specifikáció) alapján
- Metaadatot tárol típusok, crate-ek, vagy *elemek* számára
- A belső verzió az attribútumot körülvevő elemre hat (`#![inner]`)
- A külső az azt követőre (`#[outer]`)
- Léteznek unsafe attribútumok
- Sok eszköz (pl. ORM, teszt keretrendszer, kódgenerátorok) használja

Modulok

- A Rust kód modulokba szervezett
 - Modulok segítségével hierarchikusan rendszerezhető a kód
 - A modulok *elemeket* tartalmaznak (fn, struct, impl, trait, module)
 - Az elemek alapértelmezetten privát *láthatóságúak*
 - A **pub** módosítóval publikussá tehetők
 - **use** deklarációk segítségével teljes nevek rövidítéshez köthetők
 - Az **as** kulcsszóval módosítható

Modulok

- A Rust kód modulokba szervezett
 - `pub use` segítségével az elemek újraexportálhatók
 - a modulok egymásba ágyazhatók
 - a fájlhierarchia is használható beágyazásra, de deklaráció ebben az esetben is szükséges!

Kiegészítés: SSA

- A Rust programok előbb-utóbb LLVM IR-re transzformálódnak
 - Az LLVM IR SSA formában szereti a kódot
 - Még a mutable változó is ilyen kóddá alakul
- Nincs direkt összeköttetés a default immutability és az LLVM IR közt!
- Nézzük meg Compiler Explorerben az LLVM IR-t

W2T1: Próbáljuk ki a Rustdoc-ot

- Hozzunk létre egy új projektet a `cargo new` parancs segítségével
- Készítsünk benne 2 új modult, azokban néhány tetszőleges elemet
 - Készítsünk doc kommenteket `///` kezdettel
 - Próbáljuk ki a `//!` kommenteket is, pl. modulok dokumentálására
 - Használhatunk markdown-t is a kommentekben
- A dokumentáció generálásához futtassuk a `cargo doc` parancsot

W2T2: Fizzbuzz...

- A szabály a szokásos: 1..=100-ig írjuk ki a számokat, ha osztható 3-mal, **Fizz**, ha osztható 5-tel **Buzz**, ha mindkettővel, **FizzBuzz** kerüljön kiírásra a szám helyett
- Készíts legalább 2 különböző Rust szintaxist használó megoldást a FizzBuzz feladat megoldására
 - Legalább 1 megoldás használjon **match** konstrukciót (megfelelő mintával nem lesz a nyelv ellen elkövetett erőszak)

Köszönöm a figyelmet!